Computer Graphics Using OpenGL
INM376 / IN3005

Module Leader – Dr Eddie Edwards

Lecture 05: Surfaces and texture mapping

Recap from last time

- Shaders
- GLSL
- RGBA colour

Questions

- Question: When programming shaders, when does one use `uniform` vs `in`?
- Answer:
  - `uniform` is used for variables that are sent from the client (C++ program) to the GPU. Uniform variables are "visible" **in all shaders** (i.e., vertex, fragment) that are part of the shader program. Example:

client
```
pSphereProgram->SetUniform("t", m_t);
```

(any) shader stage
```
uniform float t;
```

  - Variables that are `in` are typically those coming from another shader stage (or straight from the VBO). Example:

vertex shader
```
out float fIntensity;
```

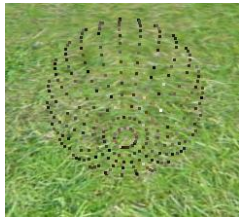
fragment shader
```
in float fIntensity;
```

Questions

- Question: Can I load a pre-compiled shader?
- Answer: Yes, however, it's not recommended. It is possible to compile a shader and load the compiled code, using `glProgramBinary`. It is unlikely that a shader compiled on one system will work on another system. It may not even work on the same system if there is a driver update. That is why it is best to compile the shaders at runtime, as done in the template code.

Questions

- Question: How does glPolygonMode work?
- Answer: glPolygonMode can change the way polygons (triangles) are rasterised. One can render the triangles as points, lines, or filled.
- Syntax is `void glPolygonMode(Glenum face, Glenum mode);`
  - *face* can be GL_FRONT, GL_BACK, or GL_FRONT_AND_BACK
  - *mode* can be GL_POINT, GL_LINE, or GL_FILL



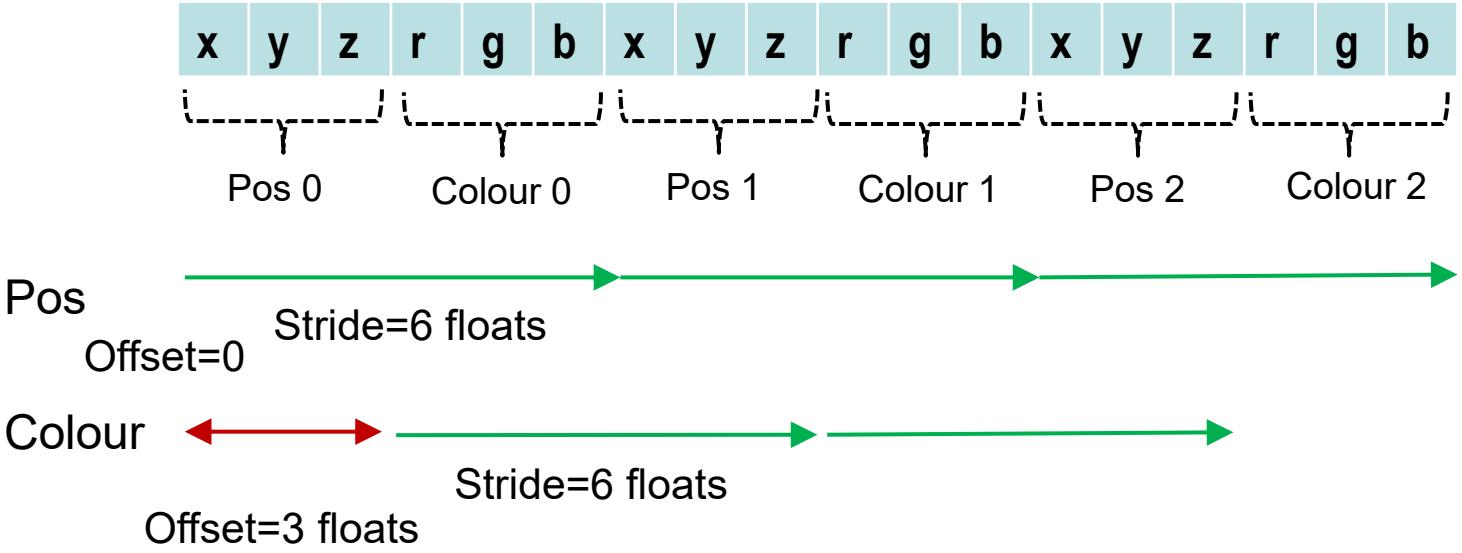
```
glPolygonMode(GL_FRONT_AND_BACK, GL_POINT);
```

```
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
```

```
glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
```

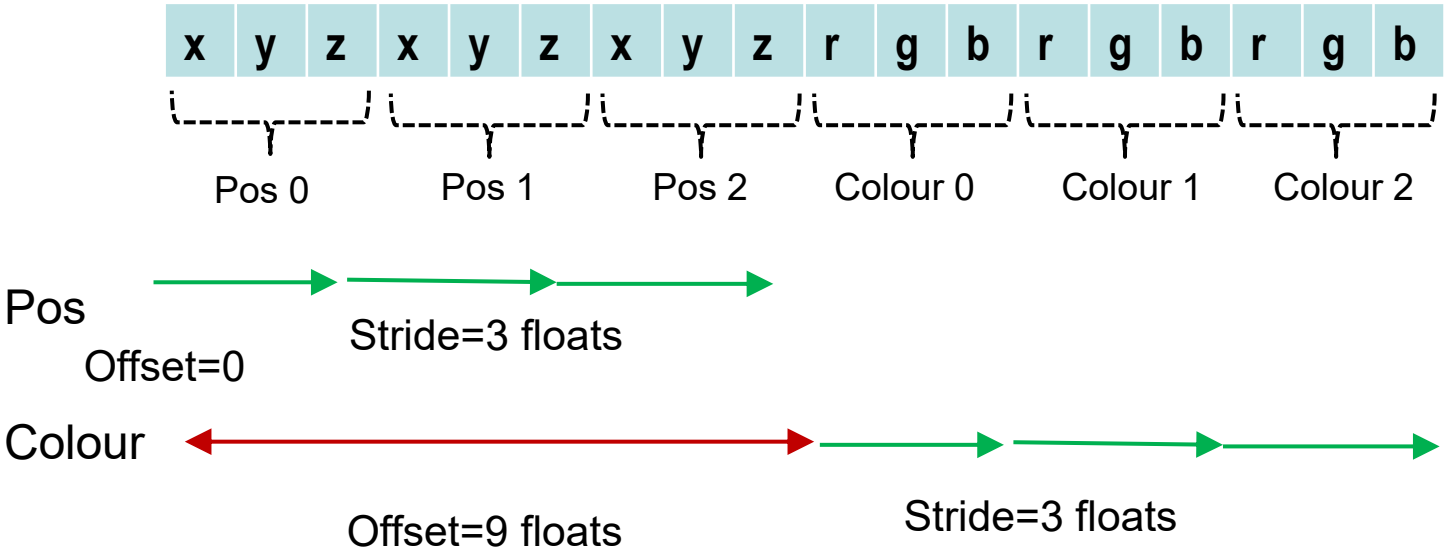
⇨ glPolygonMode is a state variable (once set, it affects everything that follows)

How does stride work? (recap from lecture 3 - interleaved)




```
// Enable position in the 0th vertex attribute
glEnableVertexAttribArray(0);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(CVertex), 0);

// Enable colour in the 1st vertex attribute
glEnableVertexAttribArray(1);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE,  sizeof(CVertex), (const GLvoid *) sizeof(glm::vec3));
```

How does stride work?
(recap from lecture 3 – tightly packed)



Stride of 0 can be confusing!
Used for tightly packed and is
the same as sizeof(element)

```
// Enable position in the 0th vertex attribute
glEnableVertexAttribArray(0);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);

// Enable colour in the 1st vertex attribute
glEnableVertexAttribArray(1);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 0, (const GLvoid *) sizeof(fTrianglePositions));
```

In the vertex shader – code is the same

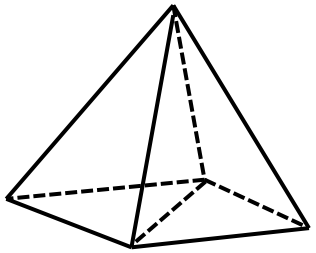
- Vertex shader:

```
#version 400

uniform mat4 modelMatrix;
uniform mat4 viewMatrix;
uniform mat4 projectionMatrix;

uniform float t;

layout (location = 0) in vec3 inPosition;
layout (location = 1) in vec3 inColour;

out vec3 vColour;

void main()
{
    gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec4(inPosition, 1.0);
    vColour = inColour;
}
```

coming from VBO

Backface culling

- OpenGL projects each face to the screen. It then determines if the front side or the back side of the face is visible.
- One can choose not to render back faces. For example, when outside an opaque closed surface, the back faces are never visible as they are occluded by nearer front faces. Therefore they can be *culled*, that is, not rendered.



How many front faces visible?

- Culling is enabled using `glEnable(GL_CULL_FACE);`
- And the side to cull is specified by `glCullFace(GLenum mode)`, where mode is
  - GL_FRONT
  - GL_BACK (default)
  - GL_FRONT_AND_BACK

⇨ Culling back faces may increase the rendering efficiency (framerate) and is enabled by default in the template.

Can I use my Macbook?

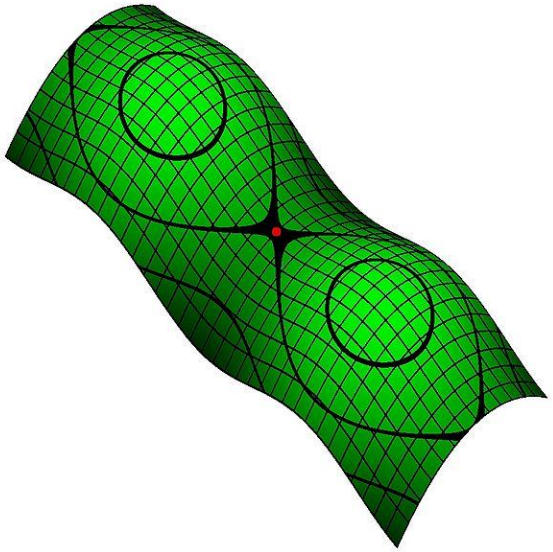
- iOS/macOS is not supported by this module.
- Compiling on Xcode is possible but requires glfw and all other libraries to be compiled for macOS (cmake is a possibility).
- Better to borrow or use a Windows machine (lab machines are fine)

Overview of this lecture

- Surfaces
- Basic texture mapping

Surfaces

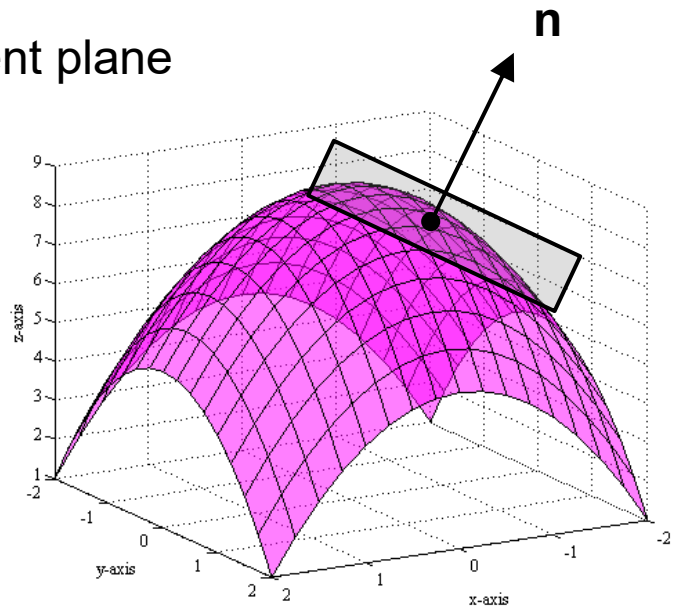
- A surface typically represents the boundary of a solid object in 3D space (R^3)
- We say a surface is a 2D *topological manifold (two-manifold)*, meaning that at a point on the surface, we can define a local coordinate patch on which a two-dimensional coordinate system is defined.
- For example, the surface of the earth is a two-manifold, with positions described by latitude and longitude.
- In computer graphics, there are different ways to represent a surface. In computer games, the typical approach is to model a surface as a collection of polygons (normally triangles).



Parametric surface

- A space curve has a parametric form **x**(t)
  - This provides a mapping from one variable (t) to R^3
- A surface has a parametric form **s**(u, v)
  - This provides a mapping from two variables (u, v) to R^3
- The derivative of a surface can be taken with respect to u and v
- These derivatives can be used to find the tangent to the surface, which is a now a plane that grazes the surface at a given point
- The surface normal **n** is a vector orthogonal to the tangent plane

$$\mathbf{s}(u, v) = [u, v, 9 - u^2 - v^2]^T$$



Exercise: surface normals of simple surfaces

- For simple surfaces, the surface normal can be identified by inspection
  - Task 1: draw the normals for the surfaces below
  - Task 2: how might you specify or compute them?

cylinder

cube

sphere

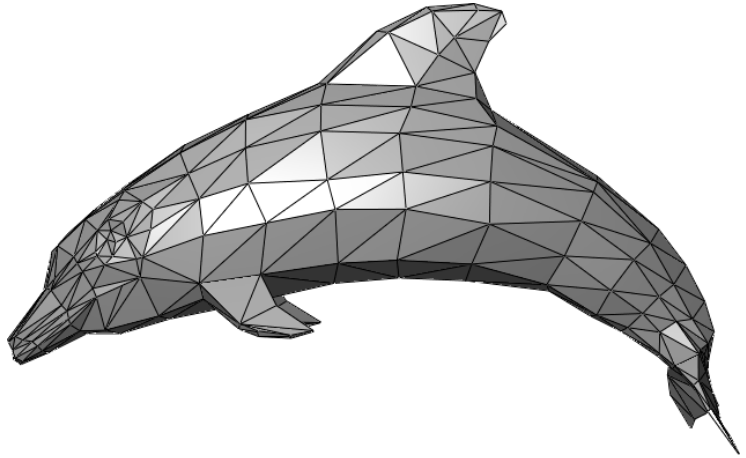
tetrahedron

Meshes

A *polygonal mesh* is a collection of vertices and their connectivity that models the surface a polyhedral object.

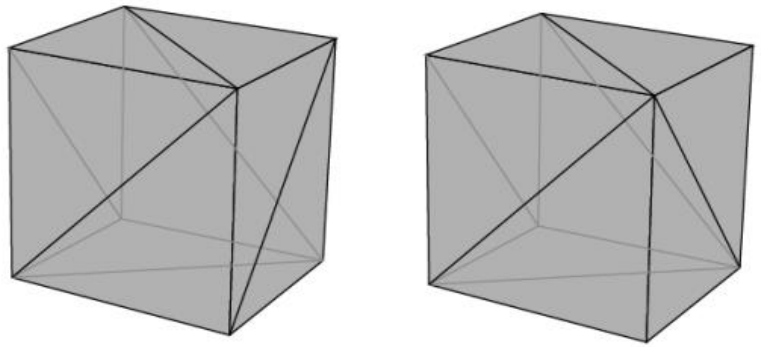
A *triangle mesh* is a polygonal mesh that is only composed of triangles.

Any polygonal mesh can be converted to a triangle mesh by splitting all polygons with more than three sides to a set of triangles.

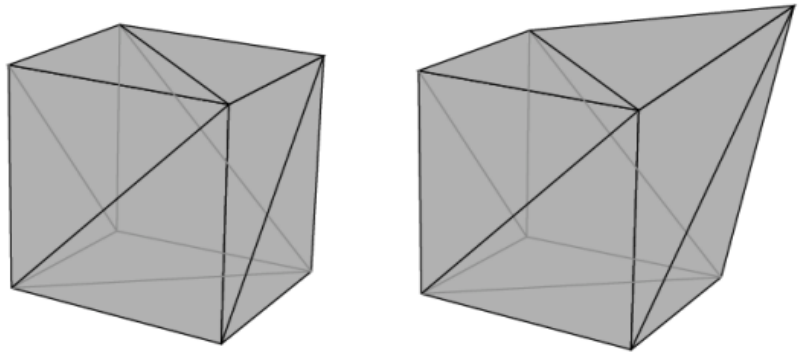


Mesh topology vs. geometry

Mesh *topology* refers to how the vertices are connected
Mesh *geometry* refers to where the vertices are in space



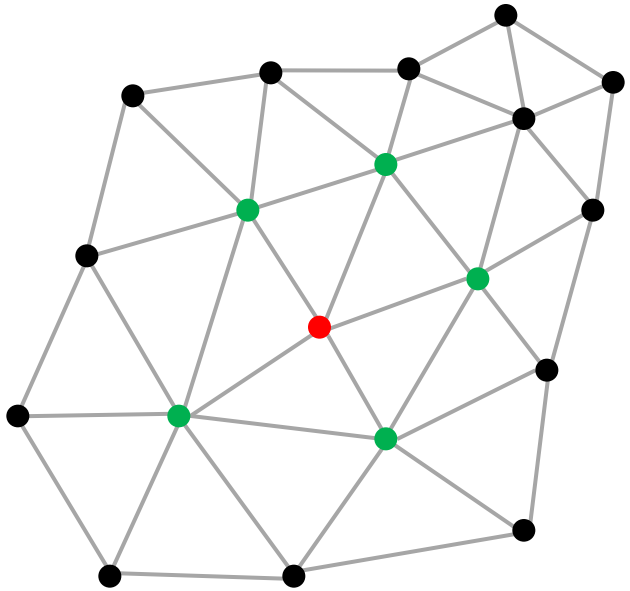
Same geometry, different topology

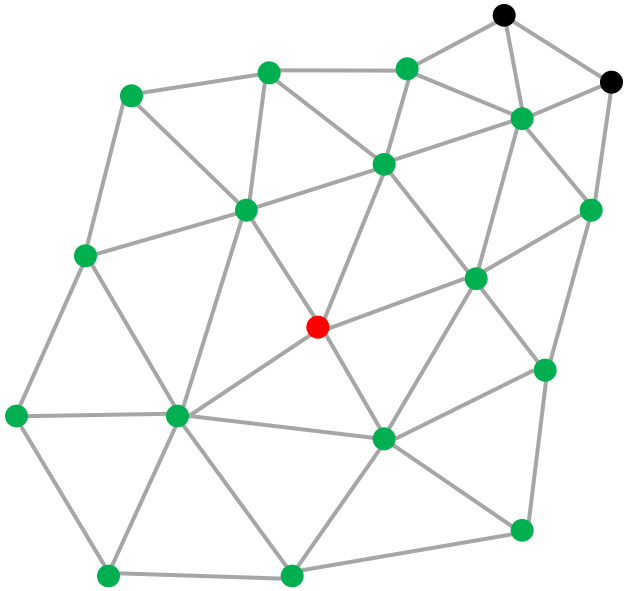


Same topology, different geometry

Neighbour vertices to a vertex

- The 1-ring neighbour vertices of a vertex **v** are vertices one edge away from **v**.
- The *k*-ring neighbour vertices of a vertex **v** is the set of *all* vertices 1 to *k* edges away from **v**



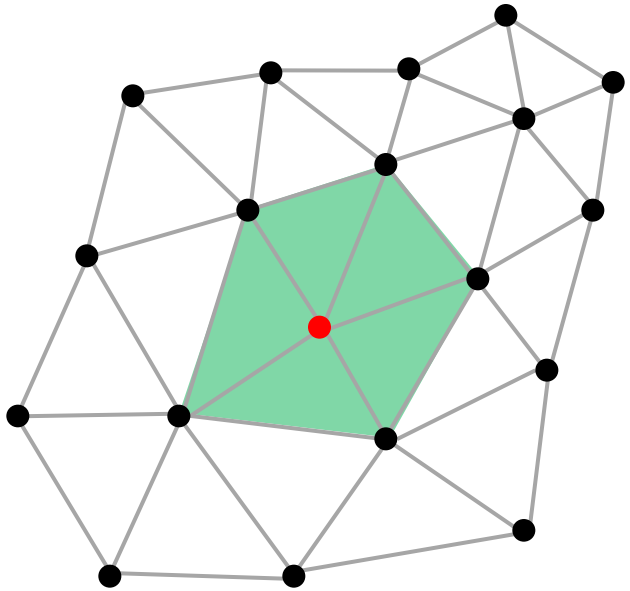


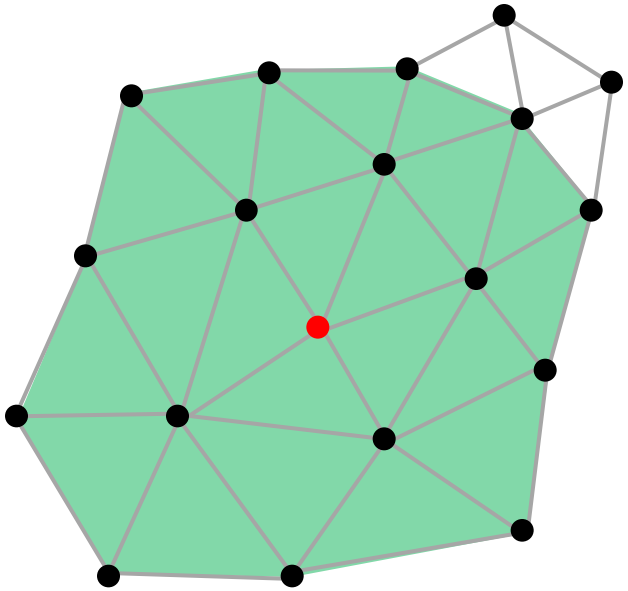
- vertex **v**
- 1-ring vertex neighbours of **v**

- vertex **v**
- 2-ring vertex neighbours of **v**

Neighbour faces to a vertex

- The 1-ring neighbour faces of a vertex **v** are faces one edge away from **v**.
- The *k*-ring neighbour faces of a vertex **v** is the set of *all* faces 1 to *k* edges away from **v**





vertex **v**

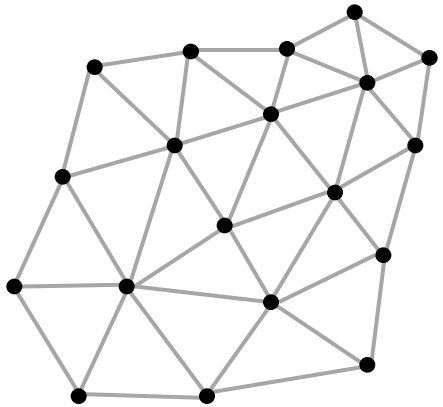
1-ring neighbour faces of **v**

vertex **v**

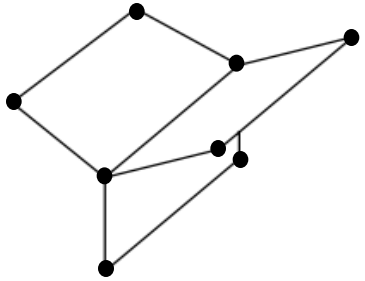
2-ring neighbour faces of **v**

Manifolds

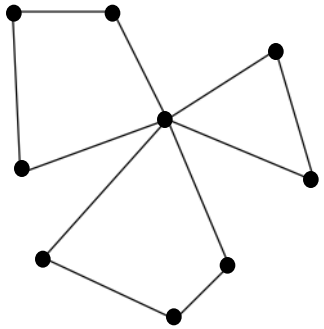
- A *manifold* is a topological space that on a small enough scale resembles Euclidean space of a given dimension.
- A mesh is a two-manifold if:
  - For a (non-boundary) vertex, the 1-ring neighbour vertices form a simple chain.
  - Each edge is shared by at most two faces.
- Common non-manifold features include surface handles and gaps.



manifold (with boundary)



non-manifold
(edge shared by 3 faces)

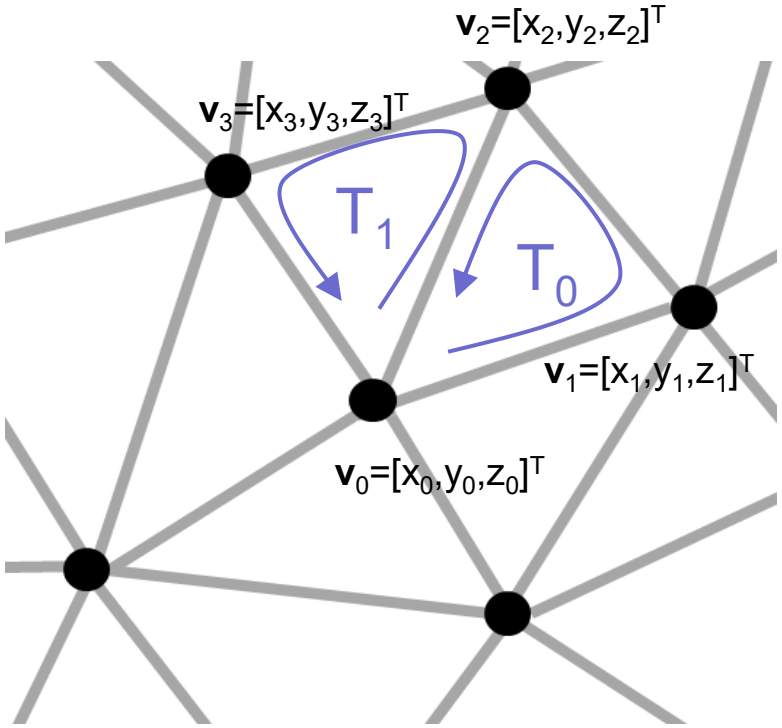


non-manifold
(1-ring neighbour vertices don't form a chain)

Triangle soup

- There are many ways to represent a mesh. The differences exist in the mesh connectivity.
- The most basic representation is *triangle soup*.
  - Separate triangles, each with its own three vertices
  - No need to explicitly model triangles – implied by every three vertices

	First vertex	Second vertex	Third vertex
T_0	x_0,y_0,z_0	x_1,y_1,z_1	x_2,y_2,z_2
T_1	x_0,y_0,z_0	x_2,y_2,z_2	x_3,y_3,z_3
	...	...	...



Triangle soup

- Triangle soup simply stores triplets of vertices in memory. Every three vertices represent a triangle.
- This approach is exceedingly simple, but has some disadvantages:
  - Many vertices are repeated – perhaps many times (once for each triangle it is in)
  - For a given triangle, it is not convenient to find neighbouring triangles or vertices.
  - Trying to search for matching vertices may be tricky due to floating point inaccuracy.

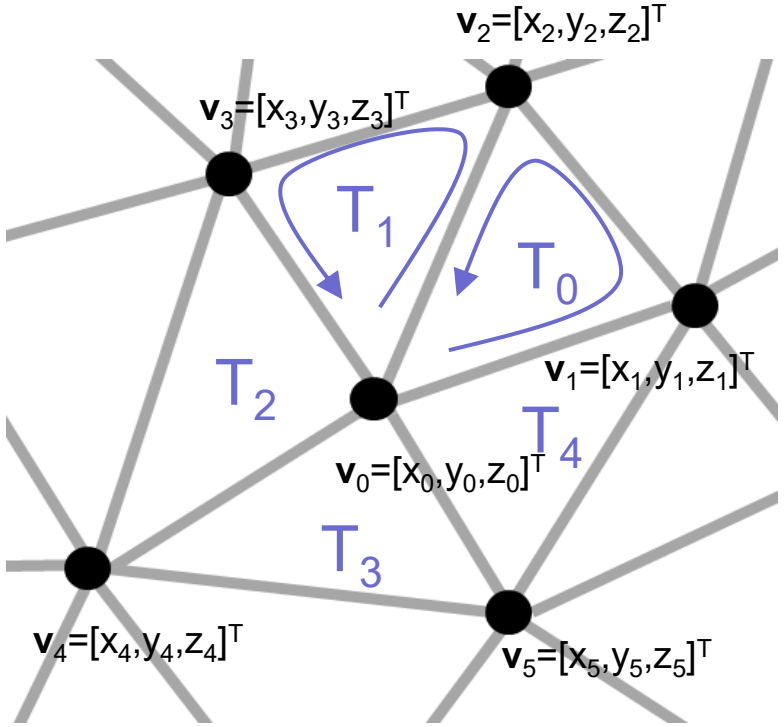
⇨ How would you find the 1-ring neighbour faces of $\mathbf{v}_0$?

Indexed mesh

- An *indexed mesh* stores:
  - a list of vertices (each vertex stored once)
  - a list triangles composed of vertex indices
- Note this separately stores the geometry and the topology

	Vertex list
$\mathbf{v}_0$	x_0,y_0,z_0
$\mathbf{v}_1$	x_1,y_1,z_1
$\mathbf{v}_2$	x_2,y_2,z_2
$\mathbf{v}_3$	x_3,y_3,z_3
$\mathbf{v}_4$	x_4,y_4,z_4
$\mathbf{v}_5$	x_5,y_5,z_5
	...

	Triangle list
T_0	**0, 1, 2**
T_1	**0, 2, 3**
T_2	**4, 0, 3**
T_3	**0, 4, 5**
T_4	**0, 5, 1**
	...



Indexed mesh

- Indexed meshes are well suited to rendering using modern graphics hardware. In particular, they can be rendered simply with the function `glDrawElements()`.
- Finding neighbours is well defined (no floating point issues) but is still inconvenient.

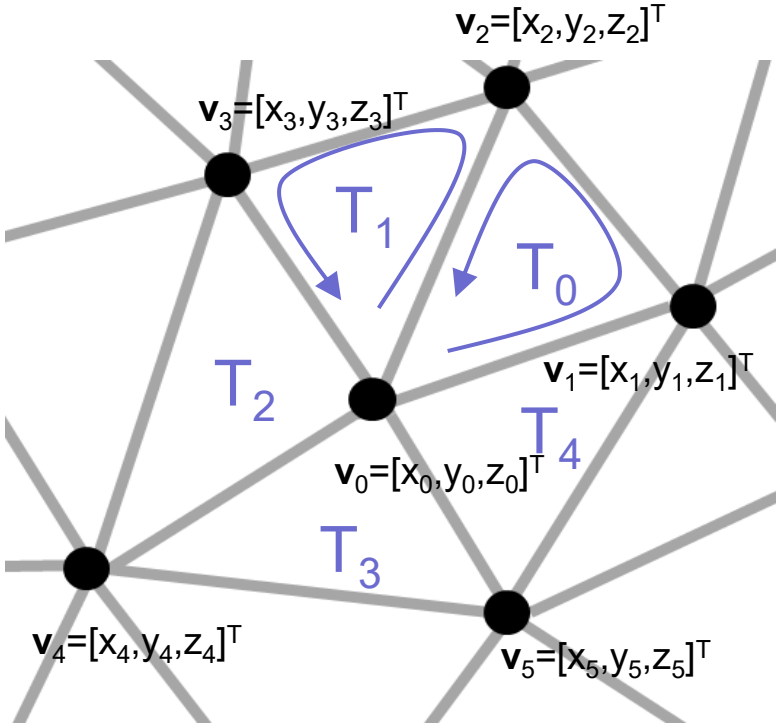
⇨ Now how would you find the 1-ring neighbour faces of $\mathbf{v}_0$?

Face vertex mesh

- A *face vertex mesh* is an indexed mesh that additionally stores, for each vertex, a list of triangles that include the vertex.

	Vertex list	On triangle
$\mathbf{v}_0$	x_0,y_0,z_0	0, 1, 2, 3, 4
$\mathbf{v}_1$	x_1,y_1,z_1	0, 4, ...
$\mathbf{v}_2$	x_2,y_2,z_2	0, 1, ...
$\mathbf{v}_3$	x_3,y_3,z_3	1, 2, ...
$\mathbf{v}_4$	x_4,y_4,z_4	2, 3, ...
$\mathbf{v}_5$	x_5,y_5,z_5	3, 4, ...
	...	...

	Triangle list
T_0	0, 1, 2
T_1	0, 2, 3
T_2	4, 0, 3
T_3	0, 4, 5
T_4	0, 5, 1
	…



⇨ Now how would you find the 1-ring neighbour faces of $\mathbf{v}_0$?

Other vertex attributes

- Typically, one stores more than positional information at a vertex. Other common vertex attributes could include
  - Colour
  - Texture coordinate
  - Normal

	Vertex list	On triangle
$\mathbf{v}_0$	$x_0,y_0,z_0,\ r_0,g_0,b_0,\ s_0,t_0,\ nx_0,ny_0,nz_0$	0, 1, 2, 3, 4
$\mathbf{v}_1$	$x_1,y_1,z_1,\ r_1,g_1,b_1,\ s_1,t_1,\ nx_1,ny_1,nz_1$	0, 4, ...
$\mathbf{v}_2$	$x_2,y_2,z_2,\ r_2,g_2,b_2,\ s_2,t_2,\ nx_2,ny_2,nz_2$	0, 1, ...
$\mathbf{v}_3$	$x_3,y_3,z_3,\ r_3,g_3,b_3,\ s_3,t_3,\ nx_3,ny_3,nz_3$	1, 2, ...
$\mathbf{v}_4$	$x_4,y_4,z_4,\ r_4,g_4,b_4,\ s_4,t_4,\ nx_4,ny_4,nz_4$	2, 3, ...
$\mathbf{v}_5$	$x_5,y_5,z_5,\ r_5,g_5,b_5,\ s_5,t_5,\ nx_5,ny_5,nz_5$	3, 4, ...
	...	...

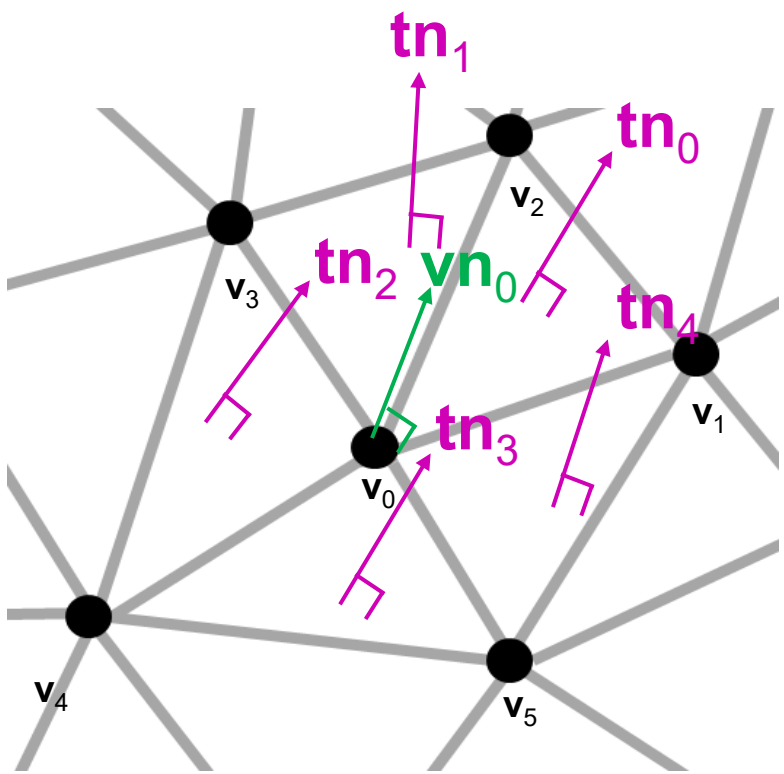
	Triangle list
T_0	0, 1, 2
T_1	0, 2, 3
T_2	4, 0, 3
T_3	0, 4, 5
T_4	0, 5, 1
	…

A possible data structure

```
class CVertex
{
    glm::vec3 m_position;
    glm::vec4 m_colour;
    glm::vec2 m_textureCoordinate;
    glm::vec3 m_normal;
};

class CTriangleList
{
    std::vector<unsigned int> m_triangleIndices;
};

class CFaceVertexMesh
{
    std::vector<CVertex> m_vertices;
    std::vector<unsigned int> m_vertexIndices;
    std::vector<CTriangleList> m_onTriangle;
};
```

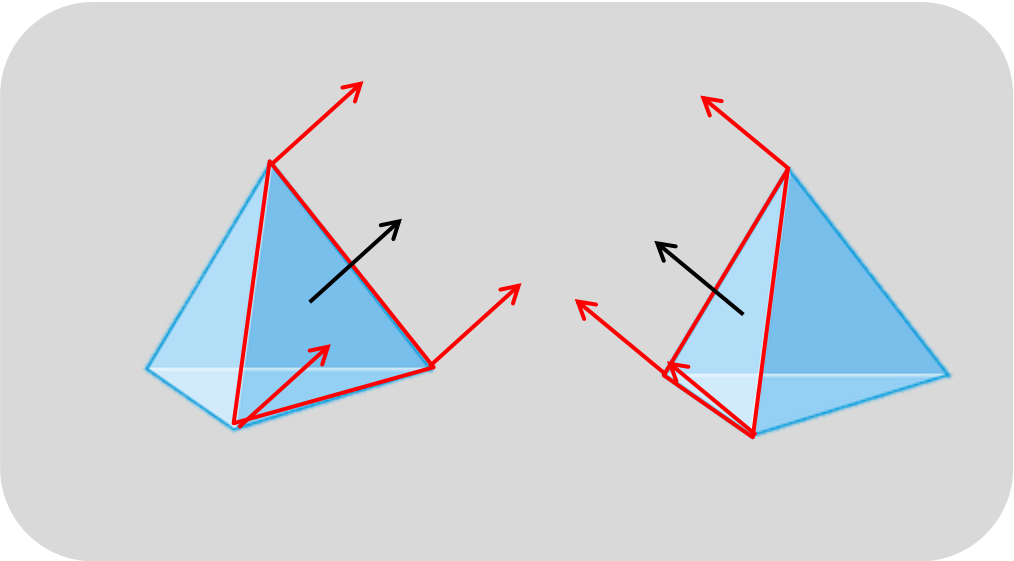
Vertex normal

- The vertex normal (**vn**) is typically the average of triangle normals (**tn**) of the one ring neighbouring faces around the vertex.
- The vertex normal is a vertex attribute.
- This is an example of why a face vertex mesh is helpful – it's very simple to compute the vertex normal since the one ring faces are readily available.



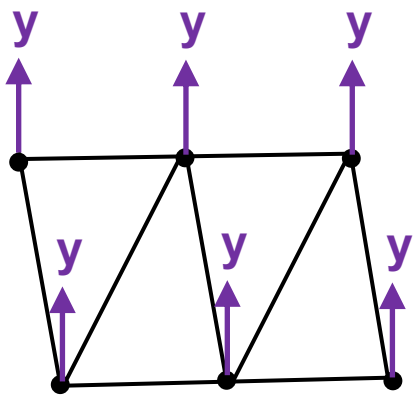
Vertex normals of simple surfaces

- Apply the surface normal to each vertex in each triangle
  - Note: a vertex may have different normals depending on the triangle.
  - Render surface as four triangles, composed of 4*3 = 12 vertices.
- Recall that two vertices with the same position but different colour, normal, or texture coordinate are *different* vertices.

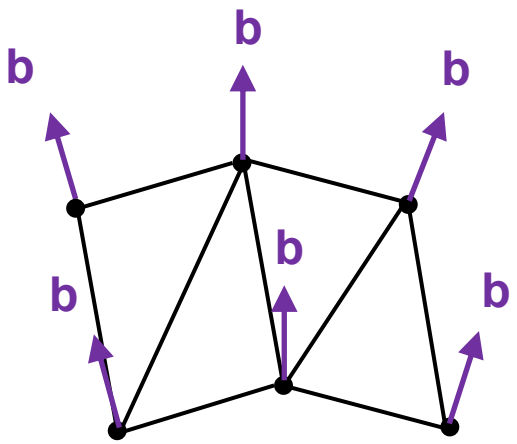


Vertex normals for a path

- If your path is planar and in the xz plane, then a natural normal for all vertices is the **y** axis.

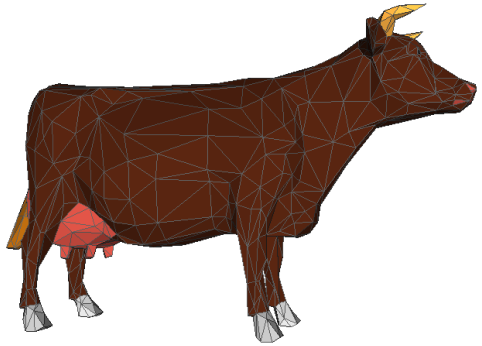


- But what if your path isn't planar?
  - The **b** axis in the TNB frame can be used instead



Mesh loaders

- There are many mesh formats. There are also many model loading libraries. Please don't reinvent the wheel! In your coursework, feel free to use any mesh format or loader you wish, including those from the internet, as long as it is compatible with your OpenGL core context. Just be sure to provide appropriate references in your coursework report.
- A few common mesh types
  - 3DS: 3D studio format
  - OBJ: Wavefront format
  - MD2… MD5: Quake model format
  - MAX: 3D studio Max format
  - X3D: XML-based format



- Open Asset Import Library (http://assimp.sourceforge.net/) is a popular loader that supports numerous mesh formats (including 3DS and OBJ) and is used in the template.

OBJ

- OBJ format has several useful properties:
  - Files are human readable (ascii text)
  - Visual Studio 2013 has a built-in OBJ mesh editor
  - Logical layout of data that is easy to follow

- However, this format does not support animation, and OBJ may be slower to load than a binary format like 3DS.

- It's possible to use a tool like Blender (www.blender.org) to convert other mesh formats to OBJ format.

- The template code uses Open Asset Import to load surfaces. If you find a mesh on the internet that doesn't load properly, try loading it in Blender, and exporting an OBJ with the following options:
  - Triangulate faces
  - Include normals
  - Include UVs (texture coordinates)
  - Write materials

Horse model in template

- Data from the horse2.obj:

```
mtllib horse2.mtl
```

← material file

```
v 0.868769 4.436345 0.464442
v 0.917819 4.410617 0.000000
…
```

← vertex positions

```
vn 0.038882 0.123175 -0.991623
vn -0.759717 0.256332 -0.597599
…
```

← vertex normals

```
vt 0.650750 0.687156
vt 0.664680 0.710709
…
```

← vertex texture coords

```
f 346/211/346 327/206/327 332/203/332
f 346/211/346 332/203/332 347/212/347
```

← triangle faces (indices for v/vt/vn)

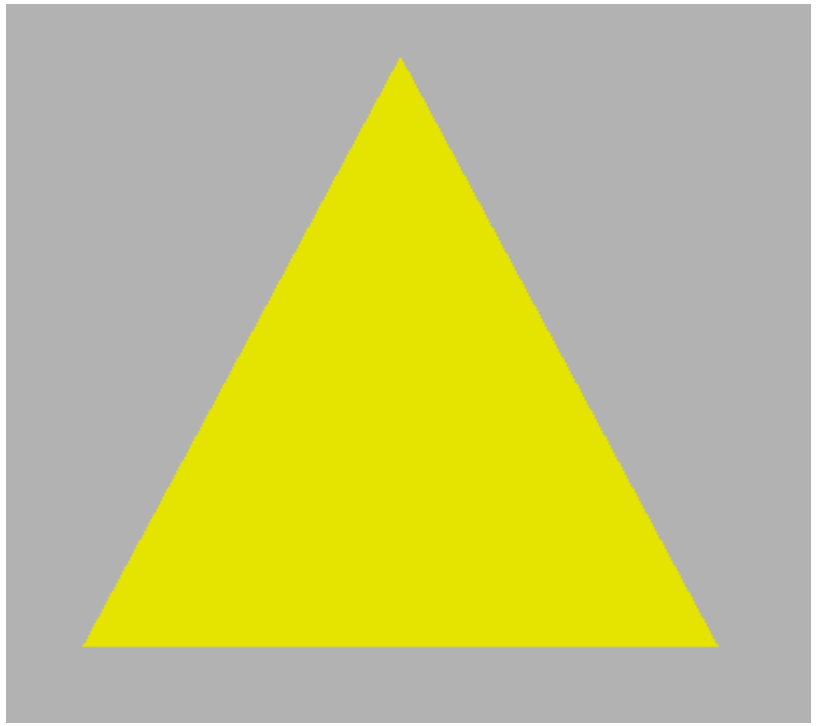
- Data from horse2.mtl:

```
map_Kd horse.bmp
```

← texture image

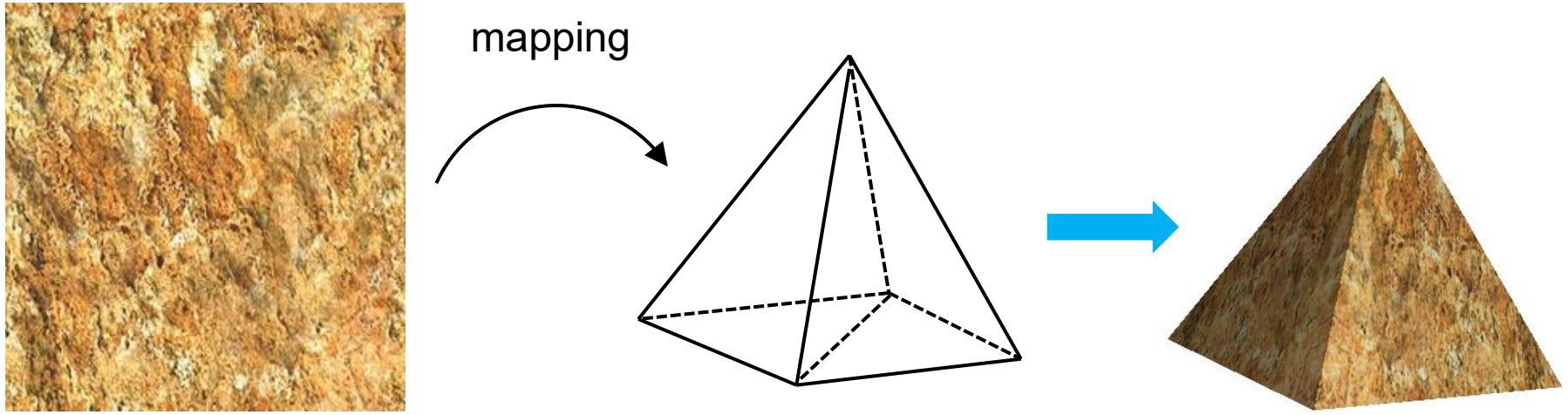
Texture mapping

- Demo



Texture mapping

- Texture mapping applies an image to a geometric primitive.



texture

(composed of *texels*)

triangles

texture-mapped triangles

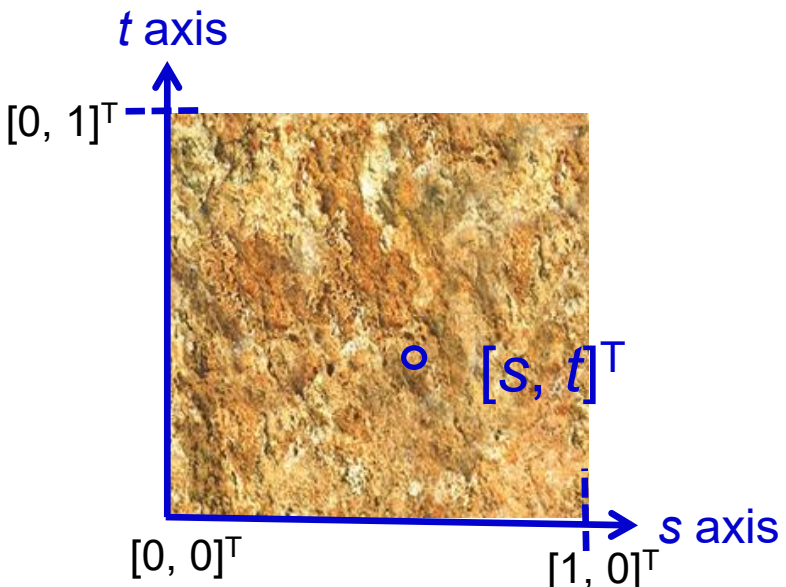
Texture mapping

- Texture mapping can dramatically increase realism without having to geometrically model fine details.
- Greg's first "law" of computer graphics: *If it looks right, it is right*
- Texture mapping can be used for many purposes, including
  - Simulating materials (bricks, wood, etc.)
  - Reducing geometric complexity (polygons)
  - Image processing
  - Reflections, shadows

⇨ The advantage of texture mapping is that visual detail is in the image, *not in the geometry*. Thus, the complexity of an image does not affect the vertex processing in OpenGL.

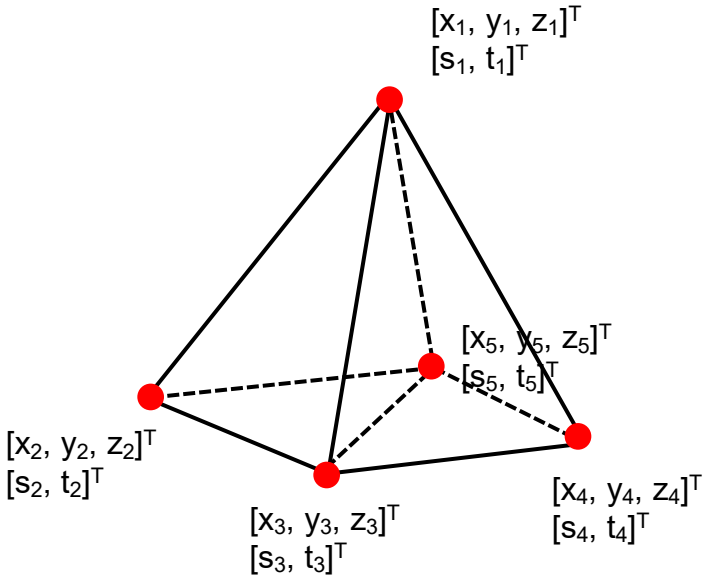
Texture coordinates

- Textures are mapped to the geometry by specifying a *texture coordinate* for each vertex.
- Texture coordinates are typically specified as floating point values that are in a range between 0.0 and 1.0.
- Texture coordinates are named *s*, *t*, *r*, and *q*:
  - *s* for 1D texture coordinates
  - $[s, t]^T$ for 2D texture coordinates (occasionally, $[u, v]^T$)
  - $[s, t, r]^T$ for 3D texture coordinates
  - $[s, t, r, q]^T$ for 4D texture coordinates



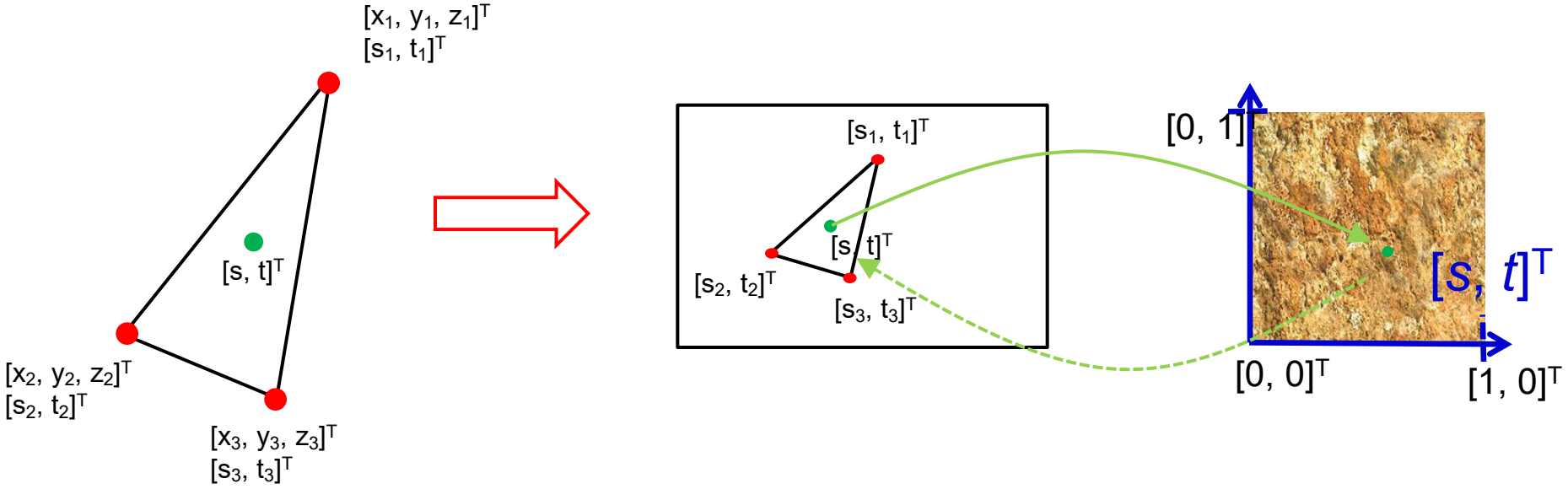
Texture coordinates

- Textures are mapped by specifying a texture coordinate for each vertex.
- OpenGL will stretch or shrink the texture as necessary to apply the texture to the model.



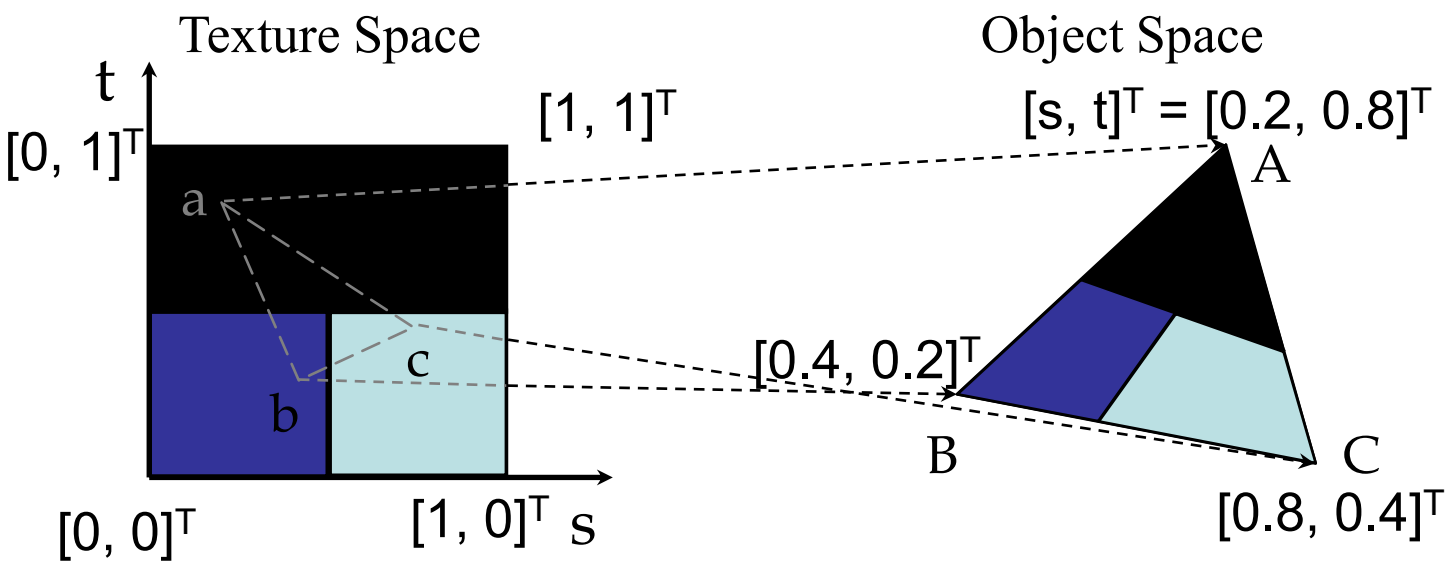
Texture coordinates

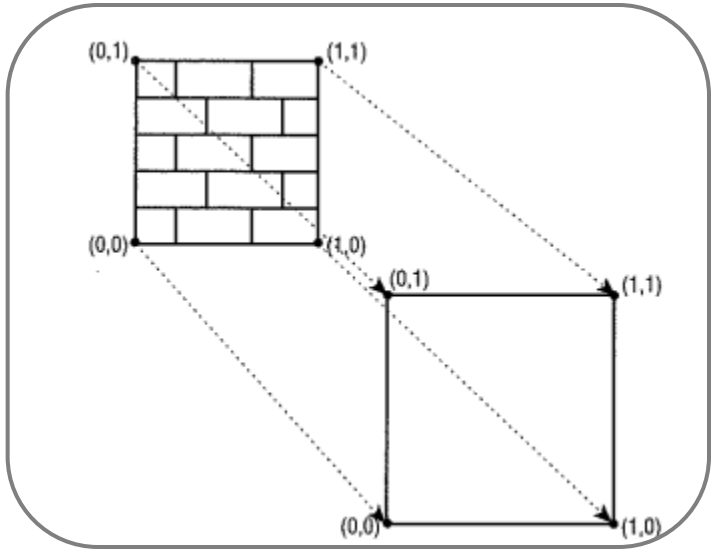
- During rasterisation, 3D primitives are projected to the camera. For each fragment, texture coordinates calculated through interpolation. Texels are looked up in the texture and applied to colour the object in the framebuffer.
- Texture coordinates are vertex attributes (like normals) that are passed to the GPU using a VBO.

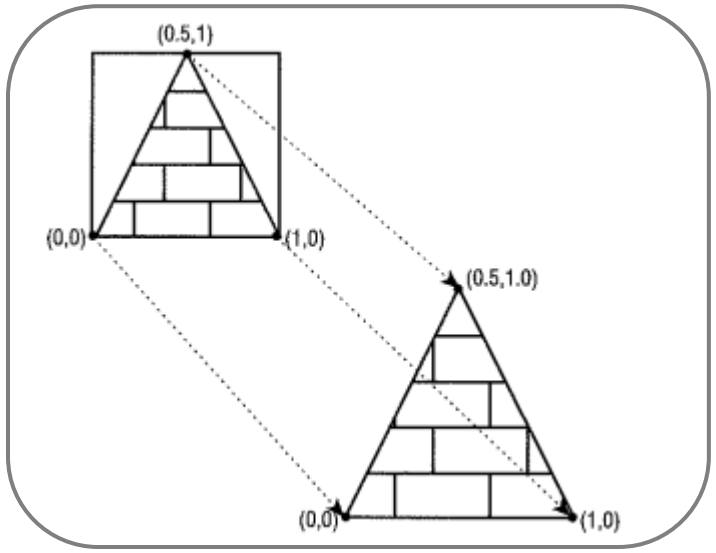


Triangle in 3D space | Frame buffer | Texel lookup

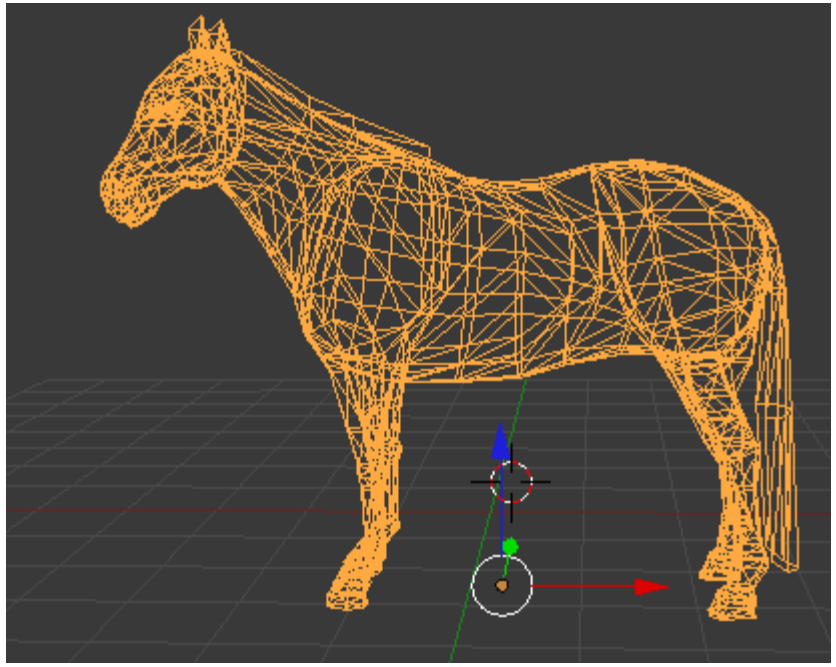
Specifying texture coordinates







Texture atlas



A *texture atlas* is a larger image that provides smaller textures for different parts of a model

Applying textures in OpenGL

Primarily done in three steps:

1. Initialisation
   - Read / generate image
   - Generate texture object
   - Transfer image to VRAM
   - Free image from RAM
   - Set texture parameters (wrapping, filtering, anisotropy, etc.)
2. Rendering
   - Bind texture object
   - Render primitives
3. Release
   - Delete texture object when it is no longer needed

⇨ In the template code includes a class `CTexture` to help with texture usage.

Generating a texture ID

- In OpenGL, it is necessary to reserve an identifier (ID) for each texture.
  - The texture ID is simply an unsigned integer
  - One image per texture ID
- The function `glGenTextures()`will request *n* texture ids and return those values back to you as `texIds.`

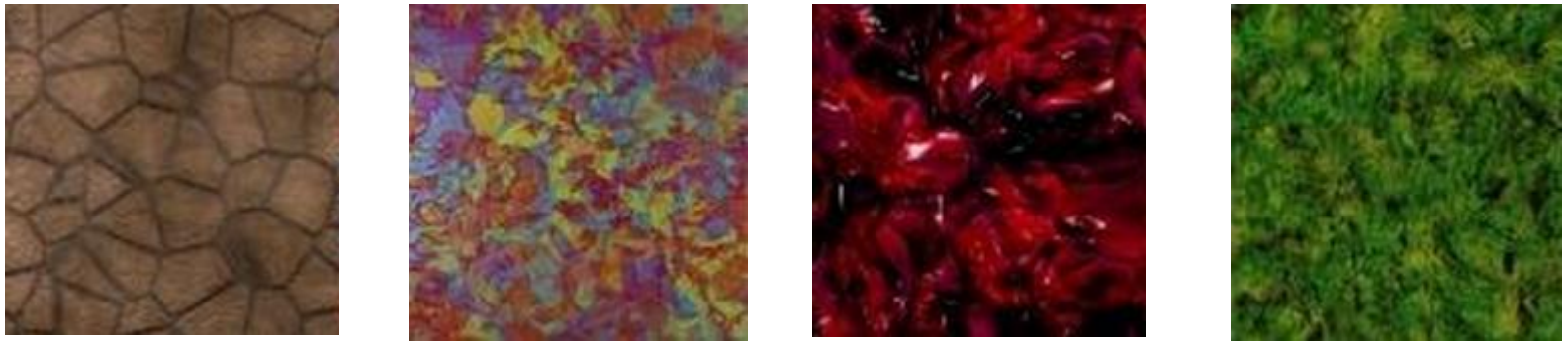
```
glGenTextures(n, *texIds);
```

Image data

- An image loader is typically used to load an image from disk and store it in RAM on the CPU side. Some common formats:
  - BMP (Windows bitmap) is an easy format and have a well published specification.
  - TGA
  - JPG
  - Many others...

⇨ The template class CTexture uses FreeImage to load images. FreeImage supports numerous formats like BMP and JPG. It is assumed colour images are 24 or 32 bit (not 8 bit).

- Alternatively, one can generate a texture procedurally (through code), e.g., using Perlin noise



Source: http://www.blender.org/development/release-logs/blender-233/procedural-textures/

Texture binding

- Typically, a computer game will use many textures.
- You can instruct OpenGL to make a particular texture current by *binding* the texture, just like you can make a particular VAO current by binding it.
- Binding is done before the texture is passed to VRAM on the graphics card. The texture is also bound during rendering, so OpenGL knows which texture to use.
- To bind a texture, the syntax is

  ```
  glBindTexture(target, id );
  ```

  - The target is typically `GL_TEXTURE_2D`
  - Once bound, the texture becomes the active texture and is used by all texturing calls until the next `glBindTexture()` is called.

- In the template code, you can bind a texture using `m_texture.Bind()`, which calls `glBindTexture()`.

Transferring the image to VRAM

- As mentioned previously, the image loader will typically load the image into RAM on the host computer (client). However, to use the texture we must pass it to the GPU (server). There are several ways to do this.
- `glTexImage2D(target, level, internalFormat, width, height, border, format, type, *texels);`
  - *target*: usually GL_TEXTURE_2D
  - *level*: 0 unless using mipmapping
  - *internalFormat* : specifies the colour components used internally by OpenGL on the GPU side (GL_RGB, GL_RGBA, etc.)
  - *width, height*: size of the image – often a power of 2
  - *border*: width of border, typically 0
  - *format* specifies the colour components on the CPU side (GL_RGB, GL_RGBA, etc.)
  - *type* defines the data type of the pixel data (e.g., GL_UNSIGNED_BYTE
  - *texels* - pointer to the client side image data.

⇨ Note that after calling glTexImage2D, the CPU side memory can be freed since the texture will be loaded into VRAM.

Texture parameters and sampler objects

- There are many different ways texturing can be configured. Texture parameters affect the rendering rules and behaviours of texture maps as they are applied. *Texture parameters* are set for the currently bound texture object.
- One way to set texture parameters is to use `glTexParameter**` calls.
- Another (**and preferred way**) is to us a *sampler object*. A *sampler object* stores sampling parameters for a texture inside a shader. It provides a more efficient way to set texturing parameters than `glTexParameter**` functions.
- Note: a *sampler object* is used to configure texture mapping parameters. It is different from a *sampler*, which is a datatype used in GLSL to access texels.

Using sampler objects

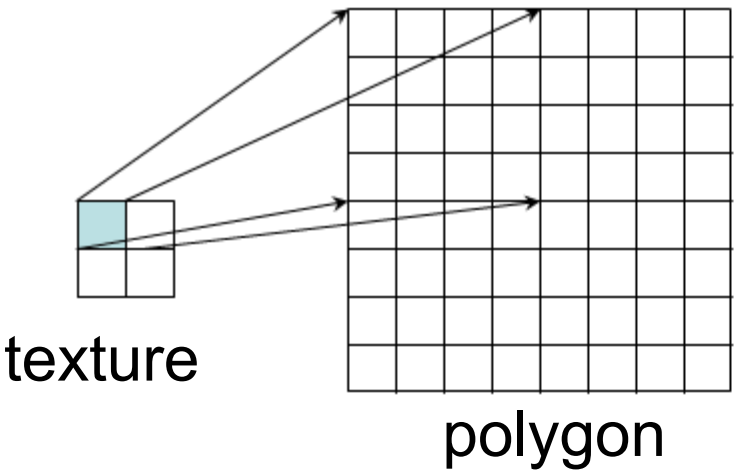
- As with other OpenGL objects, there are functions to generate, bind, and delete sampler objects:
  - `void glGenSamplers(GLsizei n, GLuint *samplers);`
  - `void glBindSampler(GLuint unit, GLuint sampler);`
  - `void glDeleteSamplers(GLsizei n, const GLuint *samplers);`

- A sampler object's parameters can then be changed using
  - `void glSamplerParameter*(GLuint sampler, GLenum pname, GLint param);`

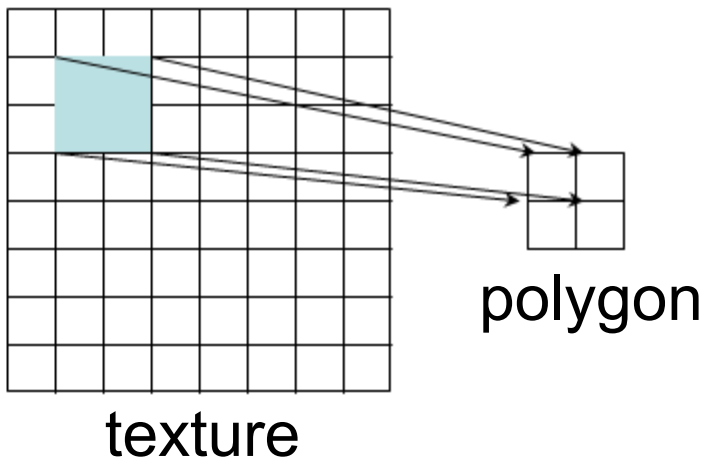
⇨ In the template code, a sampler object is created automatically for each texture as part of the `CTexture` class. A method `CTexture::SetSamplerObjectParameter[f]()` can be used to a set a parameter using the sampler object.

Texture parameters: filtering

- Textures and the 3D objects being textured are rarely the same size (in pixels). Consequently, texture images are always either stretched or shrunk (possibly both in different dimensions) when applied to geometric surfaces.
- *Texture filtering* provides magnification and minification filters for this purpose.



magnification

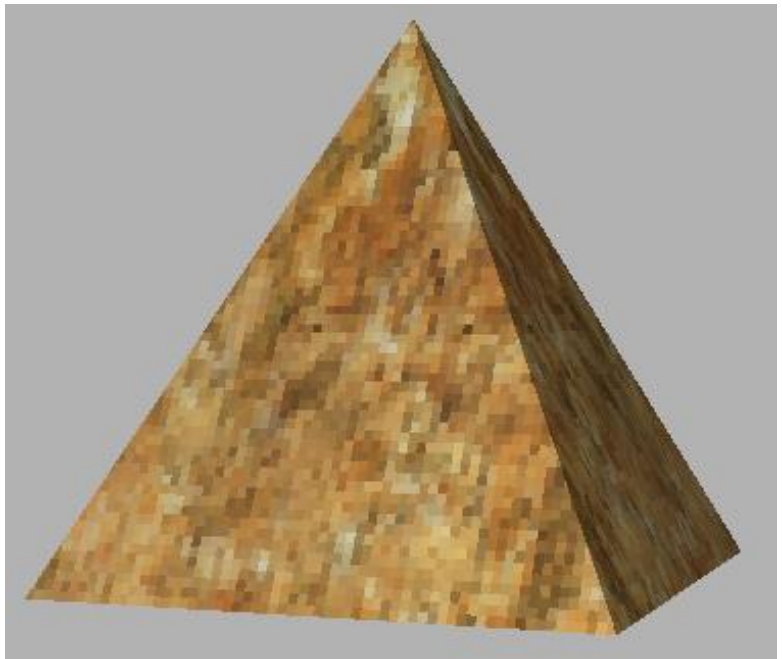


minification

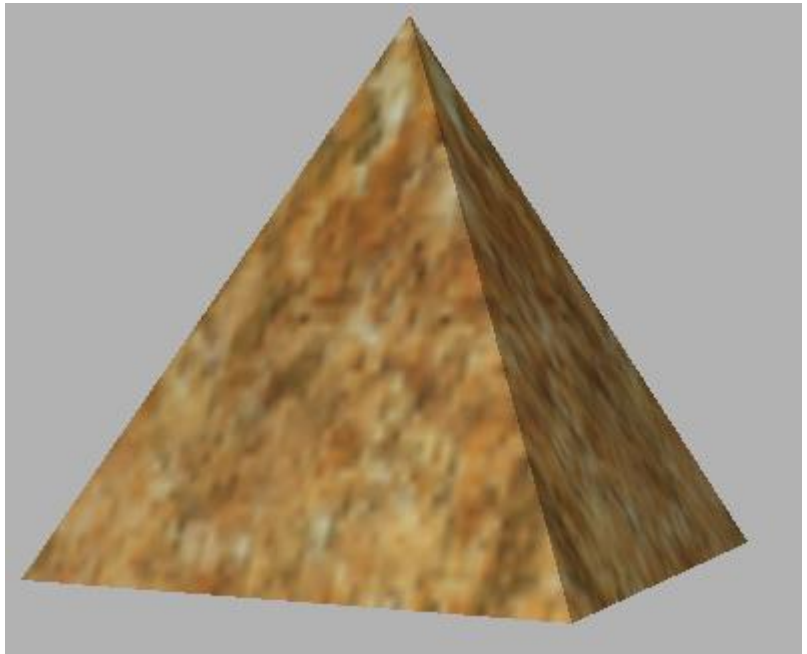
Texture parameters: filtering

- Below: nearest neighbour (left) and bilinear (right) interpolation for magnification.
- Nearest neighbour: texture coordinates are mapped into the texture. Whichever texel the coordinate lands in is chosen as the colour. May appear blocky for large magnification.
- Bilinear: Applies a weighted average of texels surrounding the mapped texture coordinate. Appearance may be blurry for large magnification but more realistic.

```
m_texture.SetSamplerObjectParameter(
 GL_TEXTURE_MAG_FILTER, GL_NEAREST);
```



```
m_texture. SetSamplerObjectParameter(
 GL_TEXTURE_MAG_FILTER, GL_LINEAR);
```

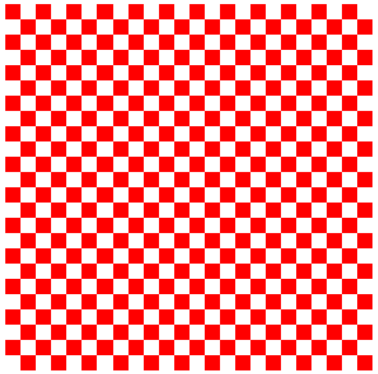


Texture parameters: wrapping modes

- Normally, one specifies texture coordinates between 0 and 1 to map to texels in the texture image.
- If texture coordinates fall out of this range, OpenGL handles them according to the *texture wrapping mode*.
- It is possible to set the wrap mode differently for *s* and *t*.

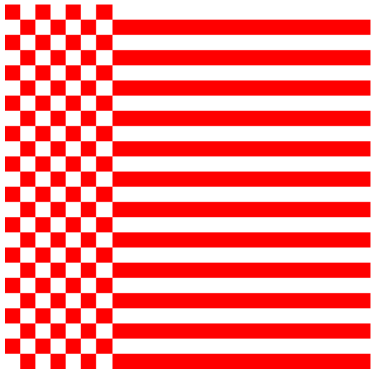
texture

Wrap S : GL_REPEAT
Wrap T : GL_REPEAT



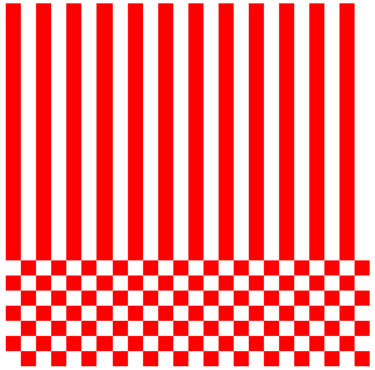
```
m_texture. SetSamplerObject
Parameter(GL_TEXTURE_WRAP_S,
GL_REPEAT);
m_texture. SetSamplerObject
Parameter(GL_TEXTURE_WRAP_T,
GL_REPEAT);
```

Wrap S :
GL_CLAMP_TO_EDGE
Wrap T : GL_REPEAT



```
m_texture.SetSamplerObject
Parameter(GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE);
m_texture.SetSamplerObject
Parameter(GL_TEXTURE_WRAP_T,
GL_REPEAT);
```

Wrap S : GL_REPEAT
Wrap T :
GL_CLAMP_TO_EDGE



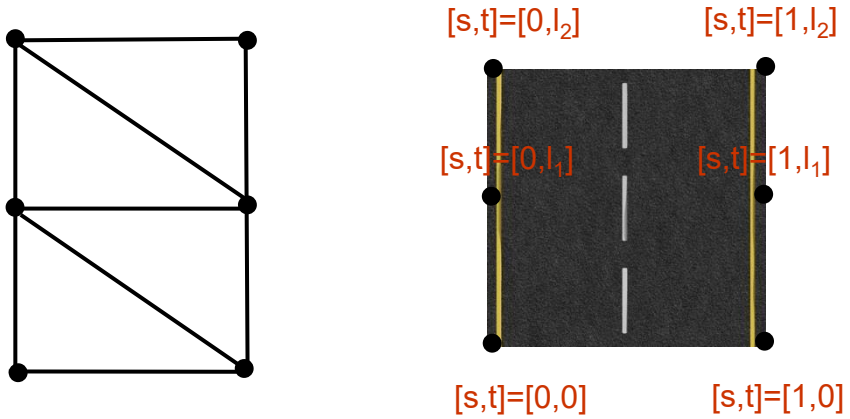
```
m_texture.SetSamplerObject
Parameter(GL_TEXTURE_WRAP_S,
GL_REPEAT);
m_texture.SetSamplerObject
Parameter(GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE);
```

Wrap S :
GL_CLAMP_TO_EDGE
Wrap T :
GL_CLAMP_TO_EDGE

```
m_texture.SetSamplerObject
Parameter(GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE);
m_texture.SetSamplerObject
Parameter(GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE);
```

Texturing a path

- To texture-map a path, it is necessary to compute appropriate values for *s* (horizontal texture coordinate) and *t* (vertical texture coordinate).



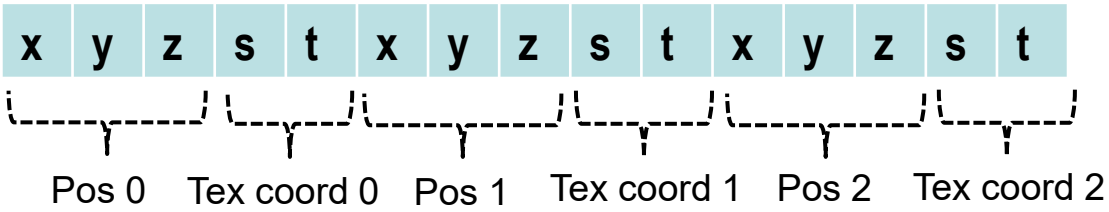
  - For *s*, it makes sense to have it 0 for the left side, and increase going to the right, to 1.
  - One can specify *t* based on the length travelled along the centreline. Values for which *t* > 1 will cause the texture to repeat (if GL_REPEAT is on).

Texture mapping: client setup

- First, we will load an image using an image loader. In the template code, there is a helper class `CTexture`. We can use it load a texture like this:

```
CTexture m_texture;
m_texture.Load(filename);
```

  - The method `Load()` will generate a texture object, bind the texture, and transfer it to the GPU
- In our VBO, we can include texture coordinates as vertex attributes. For example, interleaved attributes:



- Recall the VBO is highly configurable and could include additional vertex attributes. In the template, the VBO contains
  - Position
  - Texture coordinate
  - Normal

Texture mapping: client side

- In the fragment shader, we will use a *sampler* to read texels at different fragment locations. On the client side, we must indicate which texture unit the sampler will use. For example:

```
pMainProgram->SetUniform("sampler0", 0);
```

  - This will instruct the sampler "`sampler0`" in the fragment shader to use texture unit 0 (more on this later).

- Before rendering, we must bind the texture to make it active
  - `m_texture.Bind();`

- Rendering occurs with the function `glDrawArrays()` or one of its variants.

Texture mapping: basic shader code

Vertex shader

```
#version 400
uniform mat4 mProjection, mModelview;
layout (location = 0) in vec3 inPosition;
layout (location = 1) in vec2 inCoord;
out vec2 vTexCoord;

void main()
{
    gl_Position = mProjection * mModelview * vec4(inPosition, 1.0);
    vTexCoord = inCoord;
}
```

Fragment shader

```
#version 400
in vec2 vTexCoord;
out vec4 outputColour;
uniform sampler2D sampler0;

void main()
{
     outputColour = texture(sampler0, vTexCoord);
}
```

`vTexCoord` interpolated across primitive

Question: Would this work with an alpha-blended texture (texture with transparency)?

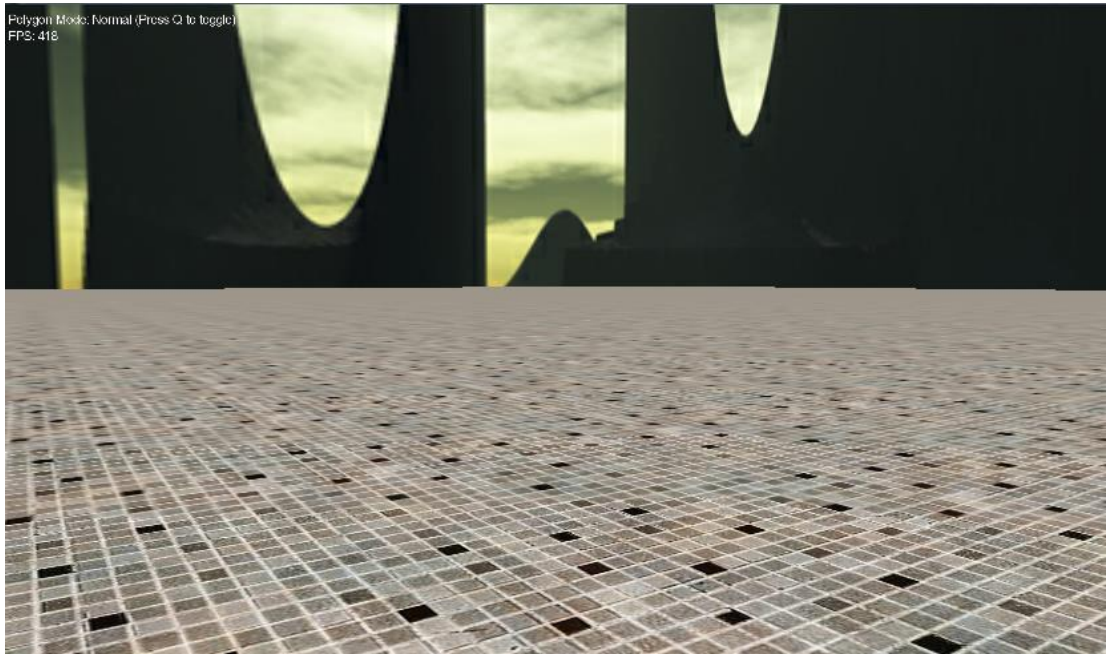
Texture mapping example

- From the template: rendering the planar ground

```
// Texture coordinates
glm::vec2 planeTexCoords[4] = {
   glm::vec2(0.0f, 0.0f),
   glm::vec2(0.0f, textureRepeat),
   glm::vec2(textureRepeat, 0.0f),
   glm::vec2(textureRepeat, textureRepeat)
};
```

`textureRepeat` repeats of the texture in each dimension

texture



Texture mapping with colour

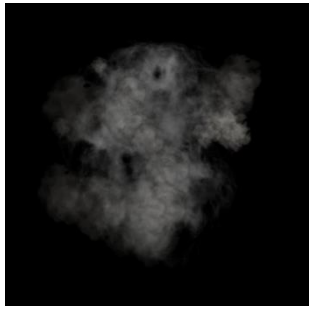
Vertex shader

```
#version 400
uniform mat4 mProjection, mModelview;
layout (location = 0) in vec3 inPosition;
layout (location = 1) in vec2 inCoord;
layout (location = 2) in vec3 inColour;

out vec2 vTexCoord;
out vec3 vColour;

void main()
{
    gl_Position = mProjection * mModelview * vec4(inPosition, 1.0);
    vColour = inColour;
    vTexCoord = inCoord;
}
```

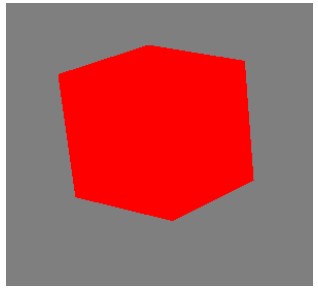
Fragment shader

```
#version 400
in vec2 vTexCoord;
in vec3 vColour;
out vec4 outputColour;
uniform sampler2D sampler0;

void main()
{
     outputColour = vec4(vColour, 1.0) * texture(sampler0, vTexCoord);
}
```

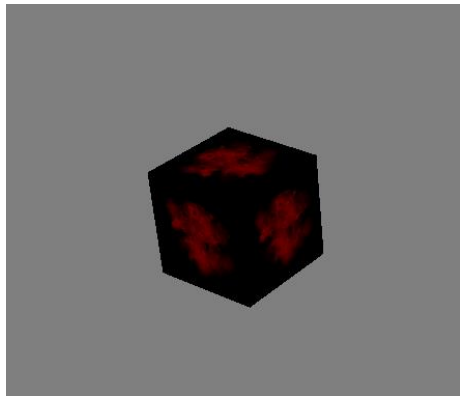
Texture mapping example



Texture applied to each face



Red cube



Combined (colour * texture)

Skybox

- As we've seen in the template, a *skybox* is a simple way to model the background of the scene using texture mapping.
- Sky, far away mountains, etc. appear in textures that are mapped to six sides of a box surrounding the camera.
- As a viewer moves through a 3D scene, the skybox remains stationary with respect to the viewer. This technique gives the skybox the illusion of being very far away since other objects in the scene appear to translate, while the skybox does not.
- The six images are carefully designed and aligned to avoid seams. It's best to render the skybox without any shading.

A skybox texture unfolded to 2D

Deletion of textures

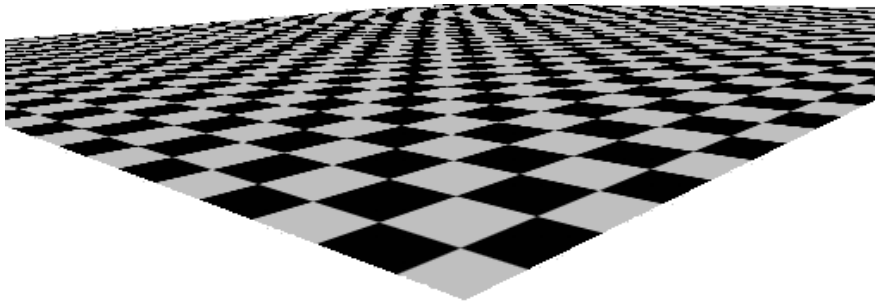
- To delete texture objects, use:

  ```
  glDeleteTextures( n, *texIds );
  ```

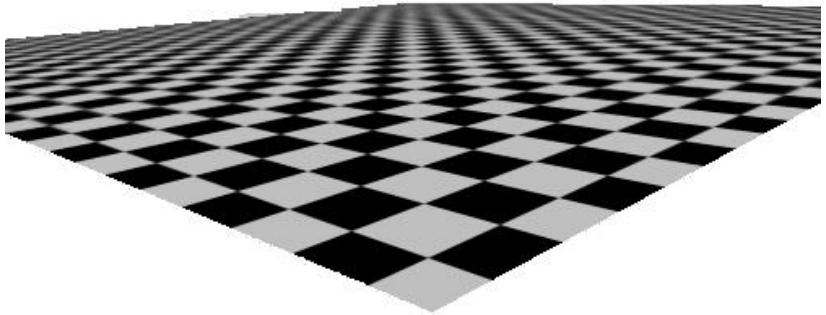
  - n is the number of texture objects to be deleted
  - *texIds* is an array of texture object identifiers to be deleted
- This will free VRAM on the graphics card.

Mipmapping

- When a large part of the texture corresponds to a single pixel in the framebuffer, simple filtering may give strong aliasing artefacts. In animations, scintillation (shimmering) artefacts may also appear.



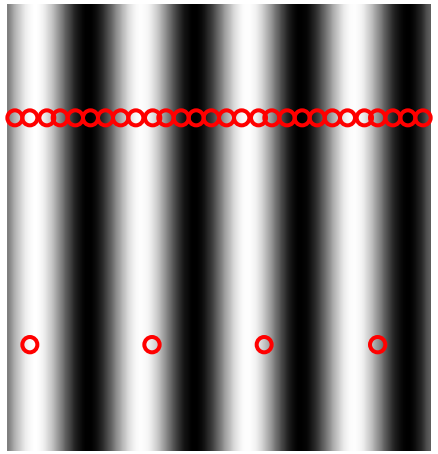
- Would prefer to average texels over a large area of the texture. One possible way to do this is to form smaller texture maps by averaging groups of texels. *Mipmapping* provides a simple way to do this.



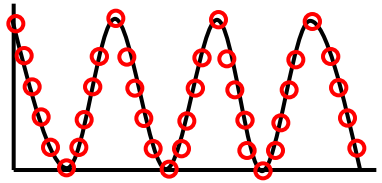
http://kineme.net/Discussion/GeneralDiscussion/unfiltration

Aside: What is aliasing?

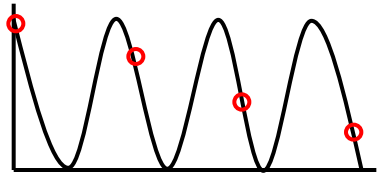
- Aliasing is an artefact that results from *undersampling a signal*. In computer graphics, it is commonly observed as
  - Spatial aliasing
  - Temporal aliasing



dense sampling



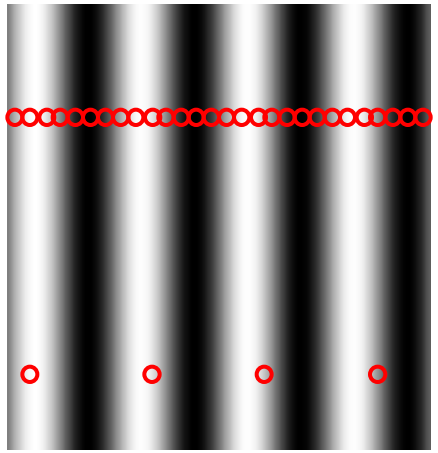
undersampling



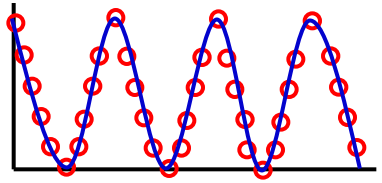
samples

Aside: What is aliasing?

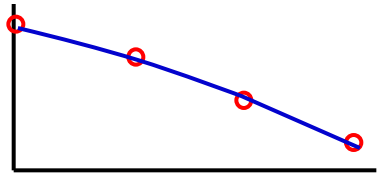
- Aliasing is an artefact that results from *undersampling a signal*. In computer graphics, it is commonly observed as
  - Spatial aliasing
  - Temporal aliasing



dense sampling



undersampling



Texture mapped primitive

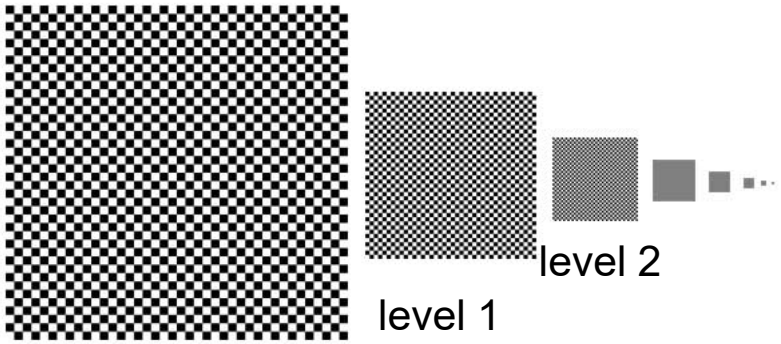
samples

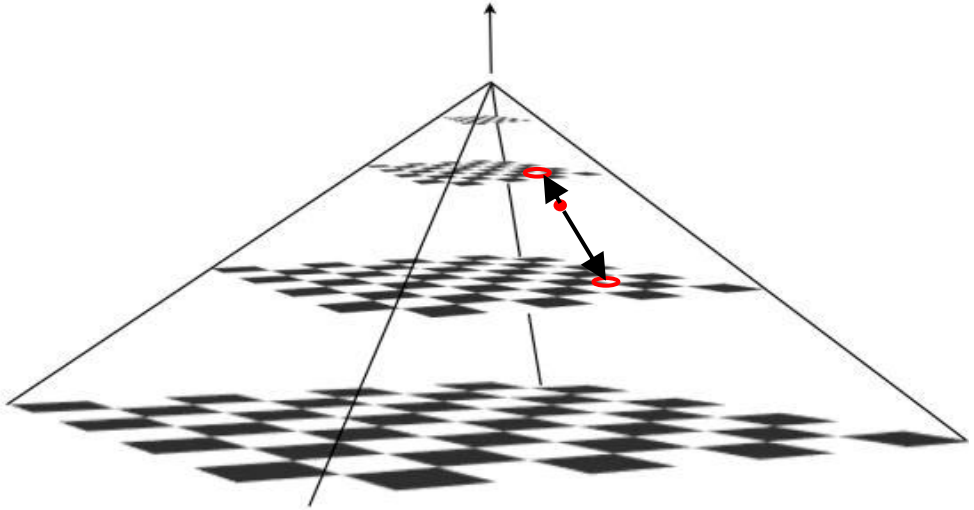
⇨ High frequencies masquerading (aliasing) as low frequencies

- Mipmaps remove the high frequencies using a stack of images

Mipmapping

- Mipmapping creates a sequence of textures, each time downsampling the image by a factor of two in width and height. The process continues until the final image is only 1x1 pixel.
- The mipmapped texture then contains multiple *levels of detail*.
- OpenGL will automatically choose the level of detail. For example, points far from the camera will be texture mapped with the smaller textures in the mipmap, while points nearer the camera are textured with larger textures.
- Mipmaps are memory efficient, only requiring 1/3rd more storage.





Example: `GL_LINEAR_MIPMAP_LINEAR`
`(trilinear interpolation)`

Mipmapping

- To create a mipmap, after calling `glTexImage2D()`, one can simply call `glGenerateMipmap()`. In the template, this happens automatically when you call `CTexture::Load(filename, mipmap=true)`, unless the 2rd argument is false.

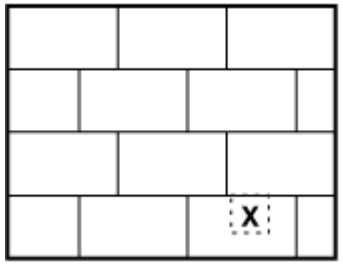
- Be sure to set the **minification** filter to one of the following modes:

Constant	Description
GL_NEAREST	Nearest neighbour filtering (NNF) on base level
GL_LINEAR	Linear filtering (LF) on base level
GL_NEAREST_MIPMAP_NEAREST	Select nearest level and perform NNF
GL_LINEAR_MIPMAP_NEAREST	Select nearest level and perform LF
GL_NEAREST_MIPMAP_LINEAR	Linearly interpolate level and perform NNF
GL_LINEAR_MIPMAP_LINEAR	Linearly interpolate level and perform LLF

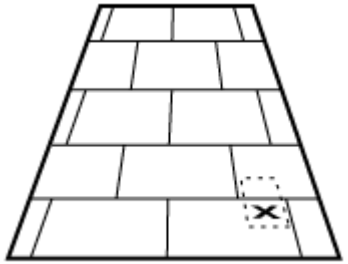
Example: `m_texture.SetSamplerObjectParameter(GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);`

Anisotropic filtering

- Anisotropic filtering is not part of the core OpenGL specification, but is a widely available extension.
- It dramatically improves the quality of texture filtering operations.
- The standard texture filtering modes use texture coordinates to average texel colours.
  - This works fine when the surface being textured is viewed orthogonal to the camera direction vector
  - At oblique views, the texture may look blurry.
- A more realistic sampling would consider the viewing angle when sampling – anistropic filtering provides this.



isotropic



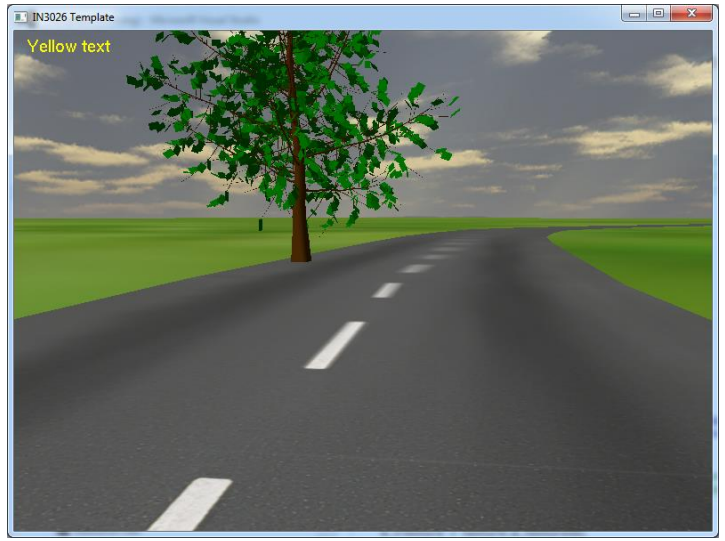
anisotropic

[OpenGL Superbible]

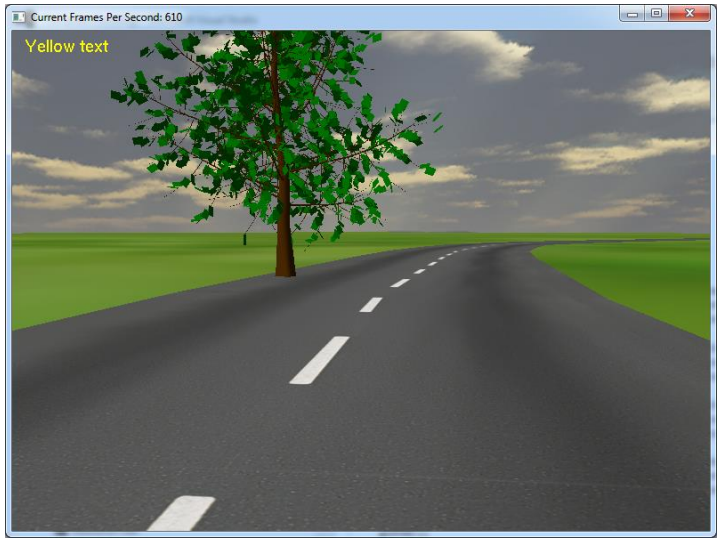
Texture parameters: aniostropy

- For a currently bound texture, anisotropic filtering can be activated as follows:~

```
GLfloat aniso;
glGetFloatv(GL_MAX_TEXTURE_MAX_ANISOTROPY_EXT, &aniso);
m_texture.SetSamplerObjectParameterf(GL_TEXTURE_MAX_ANISOTROPY_EXT, aniso);
```



Without anisotropic filtering



With anisotropic filtering

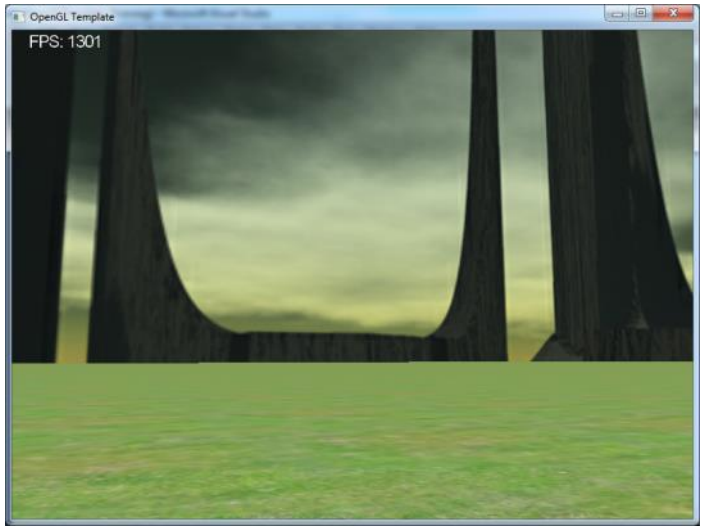
2D rendering

- OpenGL is inherently a 3D API. However, sometimes it is convenient to switch to “2D mode”, based on the screen size in pixels
- This can be achieved by changing the projection matrix using

```
glm::ortho(0, w, 0, h);
```

  where $[w, h]^T$ is the window size, and using an identity matrix for the modelview matrix.
- In the template code, there is an orthographic projection matrix based on the window size already defined. Use it, and an identity matrix for the modelview to enter “2D mode”:

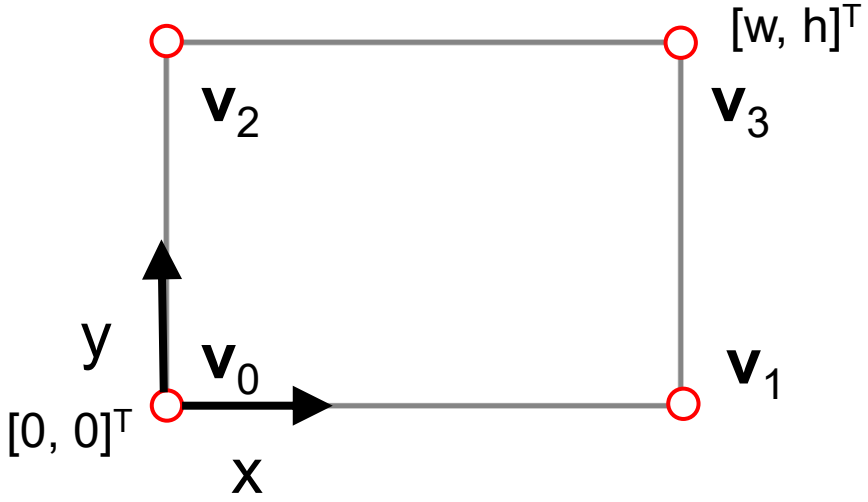
```
fontProgram->SetUniform("matrices.projMatrix",
        pCamera->GetOrthographicProjectionMatrix());
fontProgram->SetUniform("matrices.modelViewMatrix", glm::mat4(1));
```



⇨ Typically one does 2D rendering last, after all the 3D rendering has occurred. Example: in the template, the text is rendered *after* the 3D graphics.

2D rendering

- Once we're in "2D mode", units are in pixels. The OpenGL coordinate system will be



- To render a full-screen quad, simply draw a triangle strip to fill the screen (can be done with four vertices $\mathbf{v}_0$ $\mathbf{v}_1$ $\mathbf{v}_2$ $\mathbf{v}_3$). z values can be 0. For example:

```
modelViewMatrixStack.Push();
   modelViewMatrixStack.SetIdentity();
   pMainProgram->SetUniform("matrices.modelViewMatrix", modelViewMatrixStack.Top());
   pMainProgram->SetUniform("matrices.projMatrix",
         m_pCamera->GetOrthographicProjectionMatrix());
   m_pQuad->Render(); // Render quad in 2D mode
modelViewMatrixStack.Pop();
```

2D rendering

- Question: What if I want to render to part of the screen, not the full screen?
- Answer: You can use the modelview matrix to apply transformations to the quad!

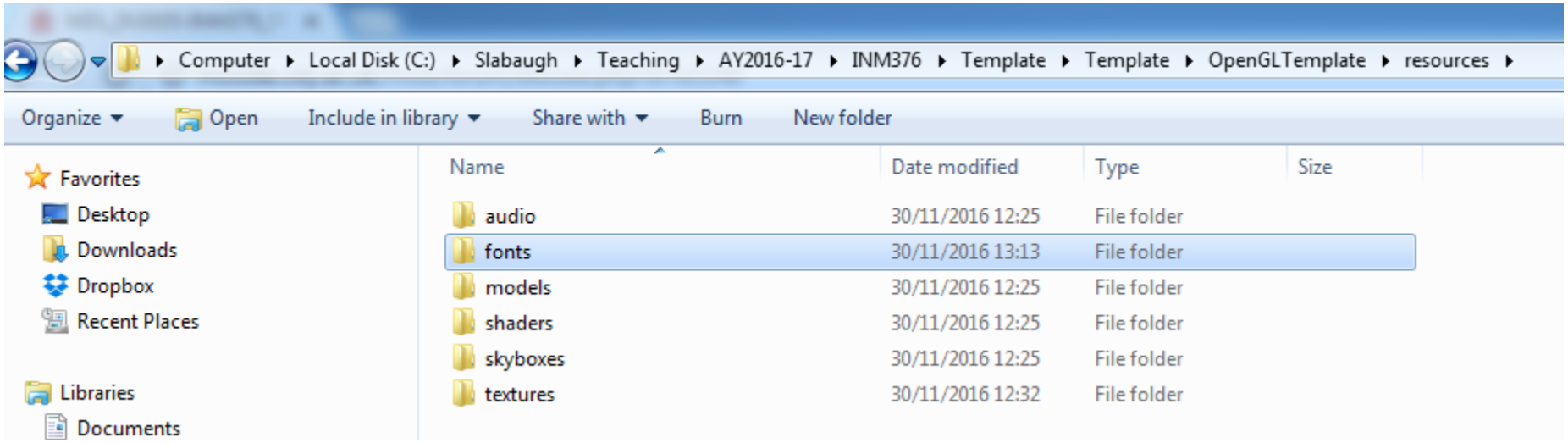
```
modelViewMatrixStack.Push();
  modelViewMatrixStack.SetIdentity();

  modelViewMatrixStack.Translate();
  modelViewMatrixStack.Rotate();        transform as needed
  modelViewMatrixStack.Scale();

   pMainProgram->SetUniform("matrices.modelViewMatrix",
modelViewMatrixStack.Top());
   pMainProgram->SetUniform("matrices.projMatrix",
         m_pCamera->GetOrthographicProjectionMatrix());
   m_pQuad->Render(); // Render quad in 2D mode
modelViewMatrixStack.Pop();
```

2D Text

- In the template, the class `CFreetypeFont` performs rendering of text.
- Create method (`LoadFont`): the class works by
  - Loading a particular font (and size)
  - It then creates small bitmap images for each character with codes 0 to 128
  - Each image is sent to the GPU as a 2D texture
  - Spacing for each character is stored
  - Position and texture coordinate data for all characters is stored in a VBO
- Render method (`Print`): Each character is rendered separately.
  - The character's texture is bound
  - A modelview matrix is translated based on the current position, and scaled based on the font size
  - A character is rendered as a quad using a triangle strip
  - The current position is advanced

Using custom fonts

- The `LoadFont` method allows you to load custom TrueType fonts (ttf files).
- If you use a custom font, it is a resource and should be included with your code.



- Then, in code, call `LoadFont` with the path to your custom font.

```
m_pFtFont->LoadFont("resources\\fonts\\totallyAwesomeFont.ttf", 32);
```

“2.5D” text

- In your game you may want to render text so that it is based on a 3D position.
- A simple way to do this is to use `glm::project` to project the 3D point to the screen and do a test against the depth buffer.
- Example: Let **q** be the 3D point where you want the text rendered.

```
GLint viewport[4];
glGetIntegerv( GL_VIEWPORT, viewport);
glm::vec4 vp(viewport[0], viewport[1], viewport[2], viewport[3]);
glm::vec3 p = glm::project(q, modelViewMatrixStack.Top(),
                  *(m_pCamera->GetPerspectiveProjectionMatrix()), vp);
GLfloat depth;
glReadPixels((int)p.x, (int)p.y, 1, 1, GL_DEPTH_COMPONENT, GL_FLOAT, &depth);
float offset = 0.001f;
if (p.z - offset < depth) {
   CShaderProgram *fontProgram = (*m_pShaderPrograms)[1];
   fontProgram->UseProgram();
   fontProgram->SetUniform("matrices.modelViewMatrix", glm::mat4(1));
   fontProgram->SetUniform("matrices.projMatrix",
         m_pCamera->GetOrthographicProjectionMatrix());
   fontProgram->SetUniform("vColour", glm::vec4(0.0f, 1.0f, 0.0f, 1.0f));
   m_pFtFont->Render(p.x, p.y, 25, "hi");
}
```

Halfway point

- How are we doing?

Interim Coursework

- How is it going?
- **DEADLINE OF 8th MARCH**
- Much of today's lecture should be of help
- We've now covered
  - Primitives
  - Geometric modeling
  - Transformations
  - Camera functions and viewing
  - Model loading
  - Texture mapping
  - Normals
  - Shaders

⇨ This covers all you need for the interim coursework.

Main Coursework

- If you haven't started yet, ***please begin as soon as possible***. The project is too complex to wait for the last few weeks to get it done, **DESPITE THE PROJECT DEADLINE OF 10th MAY**
- Start by completing the labs
- Work on both the interim and main courseworks together
- The demos have a **deadline of 7th April, presentation 11th April.**
- You should have most of your implementation completed for the demo
- Material covered so far comprises a large majority (roughly 75%) of the assignment

Coursework marking

- Increased complexity will receive higher marks
- Marking schema

Mark	Meaning
80+	Outstanding. Publishable quality. Demonstrates ability beyond module requirements.
70-79	Excellent. Work demonstrates strong understanding of material and is original.
60-69	Very competent. Key concepts understood, minor issues with implementation.
50-59	Acceptable. Learning objectives met but just enough to pass. Some technical issues.
40-49	Minimal level of achievement. Student barely grasps material. Missing components.
0-39	Unacceptable. Perhaps some understanding of material but this is outweighed by missing or incorrect material.